🧪 EXAM 8 — Recipe Management App

> **Duration:** 4 Hours **Stack:** Next.js 14+ (App Router), TypeScript, Redux Toolkit, Context API, Tailwind CSS + UI Library, Vercel Deployment

---

📋 Overview

You are tasked to build a **Recipe Management Application** using Next.js App Router. The app allows users to browse, save, create, and manage cooking recipes. Features include ingredient scaling, dietary filters, a personal cookbook (saved recipes), cooking timers, and nutritional summaries. The system must demonstrate strong component architecture, typed data models, and full-stack Next.js patterns.

📁 Application Requirements

1. Project Setup

- Initialize a **Next.js 14+** project with **TypeScript** and **App Router**.
- Configure **Tailwind CSS** and a UI library (e.g., `shadcn/ui`, `Mantine`, `Chakra UI`).
- Install **Redux Toolkit** and **React-Redux**.
- Folder structure: `app/`, `components/`, `lib/`, `store/`, `hooks/`, `types/`, `context/`.

---

2. TypeScript Types

All types must be strictly defined. `any` is forbidden.

```
// types/recipe.ts

export type DietaryTag =
  | "vegan"
  | "vegetarian"
  | "gluten-free"
  | "dairy-free"
  | "keto"
  | "paleo"
  | "nut-free";

export type Difficulty = "easy" | "medium" | "hard";

export type MeasurementUnit =
  | "g" | "kg" | "ml" | "l" | "tsp" | "tbsp"
  | "cup" | "piece" | "pinch" | "to taste";

export interface Ingredient {
  id: string;
  name: string;
  quantity: number;
  unit: MeasurementUnit;
  optional: boolean;
}

export interface RecipeStep {
  stepNumber: number;
  instruction: string;
  durationMinutes?: number;  // optional timer for this step
  tip?: string;
}

export interface Nutrition {
  calories: number;
  proteinG: number;
  carbsG: number;
  fatG: number;
  fiberG: number;
}

export interface Recipe {
  id: string;
  title: string;
  slug: string;
  description: string;
  coverImageUrl?: string;
  authorId: string;
  category: string;          // e.g., "Breakfast", "Dinner", "Dessert"
  dietaryTags: DietaryTag[];
  difficulty: Difficulty;
  servings: number;          // base serving count
  prepTimeMinutes: number;
```

```
  cookTimeMinutes: number;
  ingredients: Ingredient[];
  steps: RecipeStep[];
  nutrition?: Nutrition;     // per serving
  published: boolean;
  rating: number;            // average user rating, 0–5
  ratingCount: number;
  createdAt: string;
  updatedAt: string;
}

export interface RecipeFilters {
  category: string;
  dietaryTags: DietaryTag[];
  difficulty: Difficulty | "all";
  search: string;
  maxCookTime: number | null;   // minutes
  published: boolean | null;
}
```

---

3. App Router Structure

```
app/
├── layout.tsx                      ← Root layout (providers, nav)
├── page.tsx                        ← Public home — featured & recent recipes (server)
├── recipes/
│   ├── page.tsx                    ← Browse all recipes (server)
│   ├── loading.tsx                 ← Skeleton loading
│   ├── error.tsx                   ← Error boundary
│   └── [slug]/
│       └── page.tsx                ← Recipe detail + interactive cook mode (server + client island)
├── cookbook/
│   └── page.tsx                    ← User's saved recipe collection (client)
├── manage/
│   ├── layout.tsx                  ← Manage layout (sidebar)
│   ├── page.tsx                    ← My recipes dashboard (server)
│   ├── create/
│   │   └── page.tsx                ← Create recipe (client)
│   └── [id]/
│       └── edit/
│           └── page.tsx            ← Edit recipe (client)
└── api/
    ├── recipes/
    │   ├── route.ts                ← GET, POST
    │   └── [id]/
    │       └── route.ts            ← GET, PUT, DELETE
    └── recipes/[id]/
        └── rate/
            └── route.ts           ← POST — submit a rating
```

---

4. API Routes

`GET /API/RECIPES`

- Returns `Recipe[]`. Supports: `?category`, `?dietaryTags=vegan,keto`, `?difficulty`, `?search`, `?maxCookTime`, `?published=true`.
- Sorted by `createdAt` descending.
- Public browse always uses `?published=true`.

`POST /API/RECIPES`

- Accepts all `Recipe` fields except `id`, `slug`, `rating`, `ratingCount`, `createdAt`, `updatedAt`.
- Server generates: `id`, `slug` (from `title`, URL-safe, unique), `rating: 0`, `ratingCount: 0`, timestamps.
- Validates: `title` min 3 chars, `ingredients` at least 1, `steps` at least 1, `servings ≥ 1`, `prepTimeMinutes ≥ 0`, `cookTimeMinutes ≥ 0`.
- Returns created `Recipe` with status `201`.

`GET | PUT | DELETE /API/RECIPES/[ID]`

- Standard CRUD. `PUT` updates `updatedAt`, recomputes slug if `title` changed.

`POST /API/RECIPES/[ID]/RATE`

- Accepts `{ rating: number }` where `rating` is 1–5 integer.
- Recomputes the recipe's `rating` (running average) and increments `ratingCount`.
- Updates the recipe in the store. Returns the updated `{ rating, ratingCount }`.

---

5. Redux Toolkit — Global State

```
store/
├── index.ts
├── recipeSlice.ts
└── cookbookSlice.ts
```

`RECIPESLICE.TS`

```
interface RecipeState {
  recipes: Recipe[];
  filters: RecipeFilters;
  selectedRecipe: Recipe | null;
  status: "idle" | "loading" | "succeeded" | "failed";
  error: string | null;
}
```

Reducers: `setRecipes`, `addRecipe`, `updateRecipe`, `removeRecipe`, `setFilters`, `clearFilters`, `setSelectedRecipe`.

Thunks: `fetchRecipes`, `createRecipe`, `editRecipe`, `deleteRecipe`, `rateRecipe`.

After `rateRecipe` succeeds, dispatch `updateRecipe` with the new `rating` and `ratingCount`.

`COOKBOOKSLICE.TS`

```
interface CookbookState {
  savedIds: string[];   // IDs of saved recipes
}
```

Reducers: `saveRecipe(id)`, `unsaveRecipe(id)`, `setSavedIds(ids)`.

No async thunks needed — persistence is handled by `CookingContext` via `localStorage`.

---

6. Context API — Cooking Preferences & Cookbook

```
context/
└── CookingContext.tsx
```

```
interface CookingContextValue {
  servingMultiplier: number;           // e.g., 2.0 means double the recipe
  setServingMultiplier: (n: number) => void;
  scaleIngredient: (qty: number) => number;
  unitSystem: "metric" | "imperial";
  setUnitSystem: (s: "metric" | "imperial") => void;
  convertUnit: (qty: number, unit: MeasurementUnit) => string;
  theme: "light" | "dark";
  toggleTheme: () => void;
}
```

- `scaleIngredient(qty)` returns `qty * servingMultiplier`, rounded to 2 decimal places.
- `convertUnit(qty, unit)` converts metric units to imperial equivalents when `unitSystem === "imperial"` (e.g., grams → ounces, ml → fl oz). Returns a formatted string like `"2.5 oz"`.
- Persist `unitSystem`, `theme`, and `servingMultiplier` to `localStorage`.
- Wrap the app in this provider in `app/layout.tsx`.

---

7. Custom Hooks

`USERECIPESCALER`

```
hooks/useRecipeScaler.ts
```

- Accepts `recipe: Recipe` and `multiplier: number`.
- Returns:
  - `scaledIngredients: Ingredient[]` — a new array with each `quantity` multiplied by `multiplier` (using `CookingContext.scaleIngredient`).
  - `scaledServings: number` — `recipe.servings * multiplier`.
  - `scaledNutrition: Nutrition | undefined` — all nutrition values multiplied by `multiplier` if `recipe.nutrition` is defined.

`USERECIPEFORM`

```
hooks/useRecipeForm.ts
```

- Manages form state for creating/editing a recipe including dynamic `ingredients` and `steps` arrays.
- Accepts `initialValues?: Partial<Recipe>`.
- Returns `{ values, handleChange, errors, handleSubmit, reset, isDirty, addIngredient, updateIngredient, removeIngredient, addStep, updateStep, removeStep, moveStep }`.
- `addIngredient()` — appends a blank `Ingredient`.
- `updateIngredient(index, field, value)` — updates a specific field.

- `removeIngredient(index)` — removes at index; must keep at least 1.
- `addStep()` — appends a blank `RecipeStep` with `stepNumber = steps.length + 1`.
- `updateStep(index, field, value)` — updates step field.
- `removeStep(index)` — removes and re-numbers all remaining steps.
- `moveStep(fromIndex, toIndex)` — swaps step order and re-numbers.
- Validation: `title` ≥ 3 chars, at least 1 ingredient, at least 1 step, `servings ≥ 1`.

`USEFILTEREDRECIPES`

```
hooks/useFilteredRecipes.ts
```

- Reads recipes and filters from Redux.
- Applies: category, dietary tags (recipe must have ALL selected tags), difficulty, search (on title/description), `maxCookTime` (`prepTimeMinutes + cookTimeMinutes ≤ maxCookTime`), published flag.
- Returns `{ filteredRecipes: Recipe[], count: number, categories: string[] }`.
- `categories` is derived from the full un-filtered recipe list.

---

8. Reusable Components

`RECIPECARD`

```
interface RecipeCardProps {
  recipe: Recipe;
  variant: "public" | "manage";
  onEdit?: (recipe: Recipe) => void;
  onDelete?: (id: string) => void;
  className?: string;
}
```

- **Public**: cover image (gradient placeholder if absent), title, category, difficulty badge, total time (`prep + cook`), `StarRating`, dietary tag pills (max 3, then `+N`), save-to-cookbook heart icon.
- **Manage**: adds published/draft badge, edit and delete buttons.
- Heart icon toggles `cookbookSlice` `saveRecipe` / `unsaveRecipe` actions.

`INGREDIENTROW`

```
interface IngredientRowProps {
  ingredient: Ingredient;
  index: number;
  scaled?: boolean;            // if true, show scaled quantity from CookingContext
  editable?: boolean;
  onChange?: (index: number, field: keyof Ingredient, value: string | number | boolean) => void;
  onRemove?: (index: number) => void;
  canRemove?: boolean;
  className?: string;
}
```

- **Read mode** (`editable=false`): shows quantity (scaled if `scaled=true` via `CookingContext.scaleIngredient`), unit, name. Optional mark as "Optional".
- **Edit mode**: input fields for quantity, unit selector, name, optional checkbox. Remove button.
- Unit displays converted value when `CookingContext.unitSystem === "imperial"` using `convertUnit`.

`STEPCARD`

```
interface StepCardProps {
  step: RecipeStep;
  index: number;
  editable?: boolean;
  onChange?: (index: number, field: keyof RecipeStep, value: string | number) => void;
  onRemove?: (index: number) => void;
  onMoveUp?: (index: number) => void;
  onMoveDown?: (index: number) => void;
  isFirst?: boolean;
  isLast?: boolean;
  className?: string;
}
```

- **Read mode**: step number circle, instruction text, optional tip (collapsible), optional built-in timer (shows a countdown if `durationMinutes` is set — see cook mode below).
- **Edit mode**: text area for instruction, duration input, tip input, up/down arrows to reorder, remove button.

`NUTRITIONPANEL`

```
interface NutritionPanelProps {
  nutrition: Nutrition;
  servingMultiplier?: number;  // scale the values
  className?: string;
}
```

- Displays: Calories, Protein, Carbs, Fat, Fiber.
- Each macro has a horizontal bar showing relative proportion to Calories.
- All values multiplied by `servingMultiplier` when provided.

- Shows "per serving" label below.

`RECIPEFILTERSBAR`

```
interface RecipeFiltersBarProps {
  className?: string;
}
```

- Search input, category dropdown (from `useFilteredRecipes`), difficulty dropdown, dietary tag checkboxes (all 7 tags), max cook time slider or input.
- All changes dispatch to Redux immediately.
- Shows active filter count. "Clear All" button when any filter is set.

---

9. Recipe Detail Page (`app/recipes/[slug]/page.tsx`)

Server Component that:

- Fetches the recipe by `?slug=` (add slug query support to `GET /api/recipes`).
- If not found or `published === false`, shows a not-found UI.
- Renders: cover image, title, description, meta (difficulty, time, servings, author), dietary tags.
- Uses a **Client Component island** for:
  - **Serving size adjuster**: a `+` / `-` stepper that updates `servingMultiplier` in `CookingContext`.
  - **Unit system toggle**: metric / imperial toggle.
  - **Ingredient list**: `IngredientRow` components in read mode with `scaled={true}`.
  - **Nutrition panel**: `NutritionPanel` with the current `servingMultiplier`.
- Below the ingredient list: **Steps section** using `StepCard` in read mode.
  - Each `StepCard` with `durationMinutes` must show a **countdown timer** button:
    - Button label: `"Start Timer (X min)"`.
    - On click, starts a live countdown displayed in `MM:SS` format.
    - Timer turns red when under 10 seconds. Shows "Time's Up!" when done.
    - Only one timer can run at a time; starting a new one stops the previous.
- Below steps: a **Rate This Recipe** section — interactive `StarRating` that calls `POST /api/recipes/[id]/rate` on click and dispatches `rateRecipe` thunk.

---

10. Cookbook Page (`app/cookbook/page.tsx`)

Client Component that:

- Reads `cookbookSlice.savedIds` from Redux.
- On mount, fetches all recipes and filters to show only saved ones.
- Hydrates `savedIds` from `localStorage` on first render (via `useEffect`).

- Renders saved recipes as `RecipeCard (public variant)`.
- Shows an empty state illustration and "Browse Recipes" link when the cookbook is empty.
- Allows removing a recipe from the cookbook by clicking the heart icon.

---

11. Manage Layout & Middleware

MANAGE LAYOUT (`APP/MANAGE/LAYOUT.TSX`)

- Left sidebar: My Recipes, Create Recipe.

ROOT LAYOUT (`APP/LAYOUT.TSX`)

- Redux Provider + CookingContext Provider.
- Top nav: App name, Browse, Cookbook (with saved count badge), Manage, theme toggle.

MIDDLEWARE (`MIDDLEWARE.TS`)

- Applies to all routes under `/manage`.
- Reads a `chef_token` cookie.
- Redirects to `/?error=login_required` if missing.
- Logs path and timestamp for every matched request.

---

12. Create / Edit Recipe Pages

CREATE (`APP/MANAGE/CREATE/PAGE.TSX`) — CLIENT COMPONENT

- Uses `useRecipeForm` hook.
- Fields: title, slug (auto-generated, editable), description, category, difficulty, servings, prepTime, cookTime, dietaryTags (checkboxes), cover image URL, published toggle.
- **Dynamic ingredients list**: `IngredientRow` components in edit mode. "Add Ingredient" button calls `addIngredient`.
- **Dynamic steps list**: `StepCard` components in edit mode. "Add Step" button calls `addStep`. Up/down reorder buttons call `moveStep`.
- Optional **Nutrition** section: collapsed by default. Expanding shows fields for calories, protein, carbs, fat, fiber (all per-serving).
- On submit dispatches `createRecipe` thunk. Navigates to `/manage` on success.

EDIT (`APP/MANAGE/[ID]/EDIT/PAGE.TSX`) — CLIENT COMPONENT

- Pre-fills `useRecipeForm` from Redux `selectedRecipe` or fetched via `useEffect`.
- Same fields as create page.
- "Save as Draft" and "Publish" buttons for status control.

---

13. Data Loading Patterns

RECIPES BROWSE PAGE (`APP/RECIPES/PAGE.TSX`)

- **Server Component** with `fetch('/api/recipes?published=true', { cache: 'no-store' })`.
- Passes data to a Client Component for Redux initialization and filtering.

LOADING (`APP/RECIPES/LOADING.TSX`)

- Grid of at least 6 skeleton recipe cards with shimmer animation (image placeholder, title bar, meta row).

ERROR (`APP/RECIPES/ERROR.TSX`)

- Client Component with message and retry button calling `reset()`.

---

14. Deployment

- Deploy to **Vercel**.
- Public browse, recipe detail (with timers), cookbook, and manage sections must all function in production.
- Submit GitHub repository and Vercel URL.

🎯 Evaluation Criteria

Area	Points
TypeScript (strict, no `any`)	15
Redux Toolkit (recipe + cookbook slices)	15
Context API (CookingContext: scaling, units, theme)	10
Custom Hooks (3 hooks)	15
Reusable Components (5 components)	15
App Router & Layouts	10
API Routes (all endpoints including rating)	10
Middleware	5
Data Loading patterns	5
Vercel Deployment	5
Total	**105**

🚫 Constraints

- `any` is **not allowed** anywhere.
- Ingredient scaling must use `CookingContext.scaleIngredient` — not raw multiplication in components.
- Step reordering (`moveStep`) must correctly renumber all `stepNumber` values after every move.
- Countdown timers must be purely client-side using `setInterval` ; only one may run at a time.
- Cookbook saves must survive a page refresh (persisted to `localStorage`).
- Public GitHub repository with `README.md` required.

✅ Submission Checklist

- [ ] GitHub repository + Vercel URL
- [ ] All API routes working in production (including `POST /api/recipes/[id]/rate`)
- [ ] Redux slices for recipes and cookbook
- [ ] CookingContext with serving multiplier, unit conversion, theme — all persisted
- [ ] All 3 custom hooks (`useRecipeScaler` , `useRecipeForm` , `useFilteredRecipes`)
- [ ] All 5 reusable components (`RecipeCard` , `IngredientRow` , `StepCard` , `NutritionPanel` , `RecipeFiltersBar`)
- [ ] Create/edit pages with dynamic ingredient and step editors including reorder
- [ ] Recipe detail page with live scaling, unit toggle, and per-step countdown timers
- [ ] Cookbook page with localStorage persistence
- [ ] Middleware protecting `/manage` routes
- [ ] Skeleton loading and error boundary on browse page